Python语言与系统设计

（文件操作与应用）

Python Language & System Design

第7章 文件操作与应用

(File Operation & Applications)



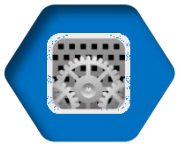
合肥工业大学 计算机与信息学院

第7章 文件操作与应用


7.1 概述


7.2 文件基本操作


7.3 数据格式化


7.4 二进制文件操作模块


7.5 文件级/目录操作


7.6 应用案例


7.7 本章小结



7.1 概述

- 文件是一个存储在辅助存储器上的数据序列，可包含任何数据内容；
- 概念上，文件是数据的集合和抽象，类似地，函数是程序的集合和抽象；
- 用文件形式组织和表达数据更有效也更为灵活。



7.1 概述

按文件中数据的组织形式

- ✓ **文本文件(Text File)**：文本文件存储的是常规字符串，由若干文本行组成，通常每行以换行符'\n'结尾。常规字符串是指记事本或其他文本编辑器能正常显示、编辑并且人类能够直接阅读和理解的字符串，如英文字母、汉字、数字字符串。文本文件可以使用字处理软件如gedit、记事本进行编辑。

- ✓ **二进制文件(Binary File)**：二进制文件把对象内容以字节串(bytes)进行存储，无法用记事本或其他普通字处理软件直接进行编辑，通常也无法被人类直接阅读和理解，需要使用专门的软件进行解码后读取、显示、修改或执行。常见的如图形图像文件、音视频文件、可执行文件、资源文件、各种数据库文件、各类office文档等都属于二进制文件。

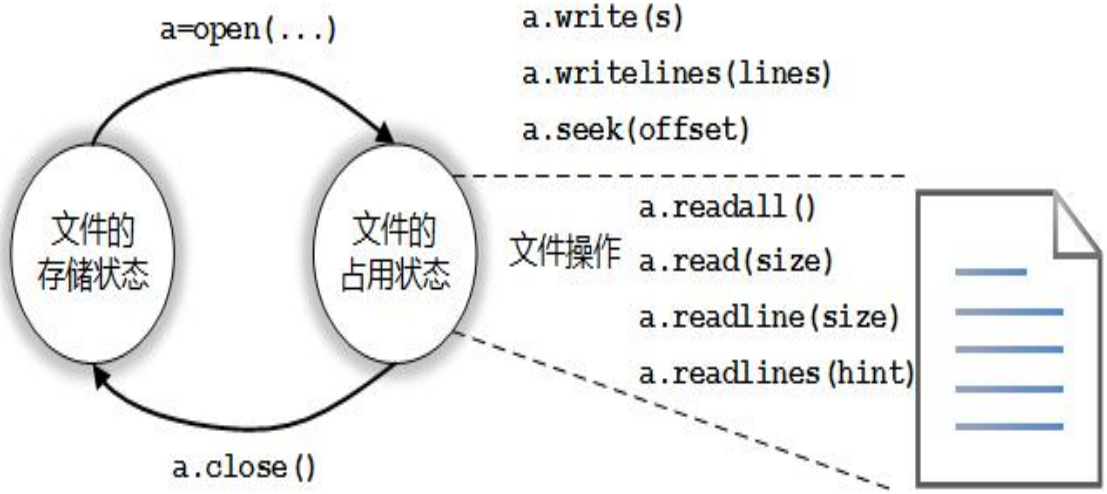
- ✓ **二进制文件：比特0和比特1组成，没统一字符编码**

- ✓ **二进制文件和文本文件最主要的区别在于是否有统一的字符编码**

ASCII码、GBK、UTF-8

7.2 文件基本操作

- 文件内容操作三步走：打开、读写、关闭。



open(file, mode = ‘r’, buffering = -1, encoding = None, errors = None,
newline = None, closefd = True, opener = None)

✓file参数指定了被打开的文件名称。
✓mode参数指定了打开文件后的处理方式。
✓buffering参数指定了读写文件的缓存模式。0表示不缓存，1表示缓存，如大于1则表示缓冲区的大小。默认值是缓存模式。
✓encoding参数指定对文本进行编码和解码的方式，只适用于文本模式，可以使用Python支持的任何格式，如GBK、utf8、CP936等等。

7.2 文件基本操作

- 如果执行正常，open()函数返回1个文件对象，通过该文件对象可以对文件进行读写操作。如果指定文件不存在、访问权限不够、磁盘空间不足或其他原因导致创建文件对象失败则抛出异常。

```
f1 = open('file1.txt', 'r')      # 以读模式打开文件
f2 = open('file2.txt', 'w')      # 以写模式打开文件
```

- 当对文件内容操作完以后，一定要关闭文件对象，这样才能保证所做的任何修改都确实被保存到文件中。`f1.close()`

- 但是，即使写了关闭文件的代码，也无法保证文件一定能够正常关闭。
  - ✓ 例如：如果在打开文件后和关闭文件前发生了错误导致程序崩溃，这时文件就无法正常关闭。
- 在管理文件对象时推荐使用with关键字，可以有效地避免这个问题。

- with语句的用法如下：

```
with open(filename, mode, encoding) as fp: #写通过文件对象fp读写文件内容的语句
```

- 上下文管理语句with还支持下面的用法：

```
with open('test.txt', 'r') as src, open('test_new.txt', 'w') as dst:
    dst.write(src.read())
```

7.2 文件基本操作

- 文件打开方式

"r"：只读模式
"w"：写入模式，会清空文件内容或创建新文件
"a"：追加模式，在文件末尾添加内容
"x"：独占模式，创建新文件，如果文件已存在则抛出异常
"b"：二进制模式
"t"：文本模式（默认模式）

7.2 文件基本操作

文件对象常用方法

方法	功能说明
close()	把缓冲区的内容写入文件，同时关闭文件，并释放文件对象
detach()	分离并返回底层的缓冲，底层缓冲被分离后，文件对象不再可用，不允许做任何操作
flush()	把缓冲区的内容写入文件，但不关闭文件
read([size])	从文本文件中读取size个字符（Python 3.x）的内容作为结果返回，或从二进制文件中读取指定数量的字节并返回，如果省略size则表示读取所有内容
readable()	测试当前文件是否可读
readline()	从文本文件中读取一行内容作为结果返回
readlines()	把文本文件中的每行文本作为一个字符串存入列表中，返回该列表，对于大文件会占用较多内存，不建议使用
seek(offset[, whence])	把文件指针移动到新的字节位置，offset表示相对于whence的位置。whence为0表示从文件头开始计算，1表示从当前位置开始计算，2表示从文件尾开始计算，默认为0
seekable()	测试当前文件是否支持随机访问，如果文件不支持随机访问，则调用方法seek()、tell()和truncate()时会抛出异常
tell()	返回文件指针的当前位置
truncate([size])	删除从当前指针位置到文件末尾的内容。如果指定了size，则不论指针在什么位置都只留下前size个字节，其余的一律删除
write(s)	把s的内容写入文件
writable()	测试当前文件是否可写
writelines(s)	把字符串列表写入文本文件，不添加换行符

7.2 文件基本操作

- 常用读写方法

读取文件：

read()：一次性读取整个文件内容

readline()：读取文件的一行内容

readlines()：读取文件的所有行并返回一个列表

写入文件：

write()：一次性写入指定内容

writelines()：写入一个字符串列表，每个字符串代表一行

文件指针操作

每个打开的文件都有一个指针，用于标识当前读写的位置。可以使用**seek()**函数来改变文件指针的位置。

删除文件

os模块的**remove()**函数来删除文件。

7.2 文件基本操作

■文件的读写

```
1 fname = input("input file:")
2 fo = open(fname, "r")
3 for line in fo.readlines():
4     print(line)
5 fo.close()
```

```
1 fname = input("input file:")
2 fo = open(fname, "r")
3 for line in fo:
4     print(line)
5 fo.close()
```

```
1 fname = input("Input FileName:")
2 fo = open(fname, "a+")
3 ls = ["合肥","工业","大学"]
4 for item in ls:
5     fo.write("\n"+item)
6
7 for line in fo:
8     print(line)
  fo.close()
```

'a' 追加写模式，文件不存在则创建，存在则在原文件最后追加内容

a+ 可读可写

fo.seek(0)



没有打印字符串？

7.2 文件基本操作

✓ 例7-1 向文本文件中写入内容，然后再读出。

```
s = 'Hello world\n文本文件的读取方法\n文本文件的写入方法\n'

with open('d:\\sample.txt', 'w') as fp:      #默认使用cp936编码
    fp.write(s)

with open('d:\\sample.txt') as fp:           #默认使用cp936编码
    print(fp.read())
```

在GBK编码中，一个汉字占2个字节

CP936其实就是GBK，IBM在发明Code Page的时候将GBK放在第936页，所以叫CP936。

✓ 例7-2 读取并显示文本文件的前5个字符。

```
with open('d:\\sample.txt', 'r') as f:
    s = f.read(5)

print('s=',s)
print('字符串s的长度(字符个数)=', len(s))
```

7.2 文件基本操作

✓ 例7-4 移动文件指针。

- seek()方法把文件指针定位到文件中指定字节的位置。但是由于对中文的支持程度不一样，当遇到无法解码的字符会抛出异常。（在GBK编码中，一个汉字占2个字节）

```
>>> s = '合肥工业大学HFUT'
>>> fp = open(r'D:\sample.txt', 'w')
>>> fp.write(s)
10
>>> fp.close()
>>> fp = open(r'D:\sample.txt', 'r')
>>> print(fp.read(3))
合肥工
>>> fp.seek(2)
2
```

```
>>> print(fp.read(1))
肥
>>> fp.seek(13)
13
>>> print(fp.read(1))
F
>>> fp.seek(3)
3
>>> print(fp.read(1))
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    print(fp.read(1))
UnicodeDecodeError: 'gbk' codec can't
decode byte 0xa7 in position 8: illegal
multibyte sequence
```

7.2 文件基本操作

- 例7-5： 读取文本文件data.txt（文件中每行存放一个整数）中所有整数，按升序排序后再写入文本文件data_new.txt中。

```
with open('data.txt') as fp:
    data = fp.readlines()
data.sort(key=int)

with open('data_new.txt', 'w') as fp:
    fp.writelines(data)
```

- 例7-6 编写程序，保存为demo6.py，运行后生成文件demo6_new.py，其中的内容与demo6.py一致，但是在每行的行尾加上了行号。

```
filename = 'demo6.py'
with open(filename, 'r') as fp:
    lines = fp.readlines()
maxLength = len(max(lines, key=len))

lines = [line.rstrip().ljust(maxLength)+'#'+str(index)+'\n'
         for index, line in enumerate(lines)]
with open(filename[:-3]+'_new.py', 'w') as fp:
    fp.writelines(lines)
```

ljust() 方法返回一个原字符串左对齐,并使用空格填充至指定长度的新字符串

```
filename = 'demo6.py'                                                    #0
with open(filename, 'r') as fp:                                          #1
    lines = fp.readlines()                                               #2
maxLength = len(max(lines, key=len))                                     #3
                                                                         #4
lines = [line.rstrip().ljust(maxLength)+'#'+str(index)+'\n'              #5
         for index, line in enumerate(lines)]                            #6
with open(filename[:-3]+'_new.py', 'w') as fp:                           #7
    fp.writelines(lines)                                                 #8
                                                                         #9
```

7.2 文件基本操作

- 例: 同时读写文本文件

```
text = '这是一段测试文本'

#以w+方式创建文件，可读可写
with open('test.txt', 'w+', encoding='utf8') as fp:
    fp.write(text)
    #定位文件指针，在utf8编码中，一个汉字占3个字节
    fp.seek(12)
    #从当前位置开始读取剩余内容
    print(fp.read())
    #重新定位
    fp.seek(12)
    #在当前位置写入新内容，覆盖原有内容
    fp.write('模拟')
```

```
测试文本
这是一个模拟文本，结束。
>>>
```

```
----------
测试文本
```

```
#重新定位
fp.seek(9)
#在当前位置写入内容
fp.write('个')
#定位到文件尾
fp.seek(0, 2)
fp.write(', 结束。')
#定位到文件头
fp.seek(0)
print(fp.read())
```

7.2 文件基本操作

- 例：批量修改记事本文件编码格式（记事本文件默认为gbk编码，改为utf8编码）

```
import os

# 获取当前文件夹中所有记事本文件清单
fns = (fn for fn in os.listdir() if fn.endswith('.txt'))
for fn in fns:
    try:
        # 首先尝试使用UTF8编码打开并读取文件内容，如果成功，就不用修改了
        # 如果失败会抛出异常
        with open(fn, encoding='utf8') as fp:
            fp.read()
    except:
        # 以默认的GBK编码读取原文件内容
        # 以UTF8编码写入新文件
        with open(fn) as fp1:
            with open('t.txt', 'w', encoding='utf8') as fp2:
                fp2.write(fp1.read())
        # 删除原文件，把新文件重命名为原文件
        os.remove(fn)
        os.rename('t.txt', fn)
```

7.3 数据格式化

数据组织的维度

- 一维数据由对等关系的有序或无序数据构成，采用线性方式组织，对应于数学中的数组和集合等概念。

中国、美国、日本、德国、法国、英国、意大利、加拿大、俄罗斯、欧盟、澳大利亚、南非、阿根廷、巴西、印度、印度尼西亚、墨西哥、沙特阿拉伯、土耳其、韩国

- 二维数据，也称表格数据，由关联关系数据构成，采用表格方式组织，对应于数学中的矩阵。

城市	环比	同比	定基
北京	101.5	120.7	121.4
上海	101.2	127.3	127.8
广州	101.3	119.4	120.0
深圳	102.0	140.9	145.5
沈阳	100.1	101.4	101.6

7.3 数据格式化

数据组织的维度

- 高维数据由键值对类型的数据构成，采用对象方式组织，属于整合度更好的数据组织方式。
- 高维数据在网络系统中十分常用，HTML、XML、**JSON**等都是高维数据组织的语法结构。

7.3 数据格式化

一二维数据的存储格式

- 逗号分割数值的存储格式叫做**CSV格式**（Comma-Separated Values，即逗号分隔值）
  - ✓ 一种通用的、相对简单的文件格式
  - ✓ 在商业和科学上广泛应用
  - ✓ 尤其应用在程序之间转移表格数据。
- **该格式的应用有一些基本规则**，例如：
  - ✓ 纯文本格式，通过单一编码表示字符；
  - ✓ 以行为单位，开头不留空行，行之间没有空行；
  - ✓ 每行表示一个一维数据，多行表示二维数据；
  - ✓ 以逗号分隔每列数据，列数据为空也要保留逗号；
  - ✓ 可以包含或不包含列名，包含时列名放置在文件第一行。

一二维数据的存储格式

■二维数据采用CSV存储后的内容如下：

```
城市,环比,同比,定基
合肥,100.6,100.1,149
北京,100.2,100.2,135.6
上海,99.9,99.8,144.7
深圳,100.5,99.3,145.9
南京,99.9,98.1,144.4
```

CSV格式存储的文件一般采用.csv为扩展名，可以通过Windows平台上的记事本或微软Office Excel工具打开，也可以在其他操作系统平台上用文本编辑工具打开。

7.3 数据格式化

一二维数据的表示和读写

- CSV文件的每一行是一维数据，可以使用Python中的列表类型表示，整个CSV文件是一个二维数据，由表示每一行的列表类型作为元素，组成一个二维列表。

```
[
['城市', '环比', '同比', '定基\n'],
['合肥', '100.6', '100.1', '149\n’],
['北京', '100.2','100.2','135.6\n'],
['上海', '99.9', '99.8', '144.7\n'],
[‘深圳’, ‘100.5’, ‘99.3’, ‘145.9\n’],
['南京', '99.9', '98.1', '144.4\n'],
]
```

7.3 数据格式化

✓ 例：导入CSV格式数据到列表

每行最后一个元素后面包含了一个换行符（"\n"）

```
fo = open("price.csv", "r")
ls = []
for line in fo:
    line = line.replace("\n","")
    ls.append(line.split(","))
print(ls)
fo.close()
```

split()函数是对字符串进行分割成列表

	A	B	C	D	E
1	城市	环比	同比	定基	
2	北京	101.5	120.7	121.4	
3	上海	101.2	127.3	127.8	
4	广州	101.3	119.4	120	
5	深圳	102	140.9	145.5	
6	沈阳	100.1	101.4	101.6	
7					
8					

✓ 例：逐行处理CSV格式数据。

```
fo = open("price.csv", "r")
ls = []
for line in fo:
    line = line.replace("\n","")
    ls = line.split(",")
    lns = ""
    for s in ls:
        lns += "{}\t".format(s)
    print(lns)
fo.close()
```

```
[['城市', '环比', '同比', '定基'], ['北京', '101.5', '120.7', '121.4'], ['上海', '101.2', '127.3', '127.8'], ['广州', '101.3', '119.4', '120.0'], ['深圳', '102.0', '140.9', '145.5'], ['沈阳', '100.1', '101.4', '101.6'], ['', '', '', '']]
>>>
```

```
城市    环比    同比    定基
北京    101.5   120.7   121.4
上海    101.2   127.3   127.8
广州    101.3   119.4   120.0
深圳    102.0   140.9   145.5
沈阳    100.1   101.4   101.6

>>>
```

7.3 数据格式化

一二维数据的表示和读写

✓ 例：一维数据写入CSV文件。

join() 方法将序列中的元素以指定的字符连接生成一个新的字符串

'北京','101.5','120.7','121.4\n'

```
1  fo = open("price1.csv", "w")
2  ls = ['北京', '101.5', '120.7', '121.4']
3  fo.write(",".join(ls)+ "\n")
4  fo.close()
```

	A	B	C	D	E
1	北京	101.5	120.7	121.4	
2					
3					

■ 对于列表中存储的二维数据，可以通过循环写入一维数据的方式写入CSV文件，参考代码样式如下：

```
for row in ls:
    <输出文件>.write(",".join(row)+"\n")
```

高维数据的格式化

如果要将一个系统内的数据通过网络传输给其它系统或客户端，需要先把这些数据转化为字符串或字节串，而且需要规定一种统一的数据格式才能让数据接收端正确解析并理解这些数据的含义。XML 是早期被广泛使用的数据交换格式，如今大家使用更多的数据交换格式是JSON（JavaScript Object Notation），它是一种轻量级的数据交换格式。

- JSON格式可以对高维数据进行表达和存储。
- **JSON（JavaScript Object Notation）是一种轻量级的数据交换格式，易于阅读和理解。**
- JSON格式表达键值对<key, value>的基本格式如下，键值对都保存在双引号中：

"key" : "value"

当多个键值对放在一起时，JSON的规则：

- 数据保存在键值对中；
- 键值对之间由逗号分隔；
- 括号用于保存键值对数据组成的对象；
- 方括号用于保存键值对数据组成的数组。

以“本书作者”JSON数据为例。

```
"本书作者":[
{  "姓氏":"嵩",
"名字":"天",
"单位":"北京理工大学"  },
{          "姓氏":"礼",
"名字":"欣",
"单位":"北京理工大学"  },
{  "姓氏":"黄",
"名字":"天羽",
"单位":"北京理工大学"  }
]
```

- json库主要包括两类函数：**操作类函数和解析类函数**
  - 操作类函数主要完成外部JSON格式和程序内部数据类型之间的转换功能
  - 解析类函数主要用于解析键值对内容。

dumps()和loads()分别对应编码和解码功能。

函数	描述
json.dumps(obj, sort_keys= False, indent=None)	将Python的数据类型转换为JSON格式，编码过程
json.loads(string)	将JSON格式字符串转换为Python的数据类型，解码过程
json.dump(obj, fp, sort_keys= False, indent=None)	与dumps()功能一致，输出到文件fp
json.load(fp)	与loads()功能一致，从文件fp读入



7.3 数据格式化

使用JSON序列化-简称 编码-解码

- Python标准库json完美实现了该格式，用法类似于marshal和pickle。

```
>>> import json
>>> json.dumps(['a','b','c'])  # 序列化列表对象，直接查看序列化后的结果
'["a", "b", "c"]'

>>> json.loads(_)                  # 反序列化
['a', 'b', 'c']
>>> json.dumps({'a':1, 'b':2, 'c':3})     # 序列化字典对象
'{"a": 1, "b": 2, "c": 3}'
>>> json.loads(_)
{'a': 1, 'b': 2, 'c': 3}
```

将对象转换为可通过网络传输或可以存储到本地磁盘的数据格式（如：XML、JSON或特定格式的字节串）的过程称为序列化；反之，则称为反序列化。

在 json 的编解码过程中，Python 的原始类型与 json 类型会相互转换

```
>>> type(s1)
<class 'str'>
>>> type(dt)
<class 'dict'>
```

```
>>>import json
>>>dt = {'b':2,'c':4,'a':6}
>>>s1 = json.dumps(dt)
          #dumps将python类型转换为JSON格式的字符串类型
>>>s2 = json.dumps(dt,sort_keys=True,indent=4)
          #indent表示缩进级别，sort_keys按键值排序,默认False
```

```
>>>print(s1)
{"c": 4, "a": 6, "b": 2}
>>>print(s2)
{
    "a": 6,
    "b": 2,
    "c": 4
}
>>>print(s1==s2)
False
>>>dt2 = json.loads(s2) #将JSON格式转换为python类型
>>>print(dt2, type(dt2))
{'c': 4, 'a': 6, 'b': 2} <class 'dict'>
```

如果要处理的是文件而不是字符串，可以使用 json.dump() 和 json.load() 来编码和解码JSON数据

```
# 写入 JSON 数据
import json
data = {'b':2,'c':4,'a':6}

with open('data.json', 'w') as f:
    json.dump(data, f)

# 读取数据
with open('data.json', 'r') as f:
    data = json.load(f)
```

7.4 二进制文件操作模块

二进制文件操作

- 数据库文件、图像文件、可执行文件、音视频文件、Office文档等等均属于二进制文件。
- 对于二进制文件，不能使用记事本或其他文本编辑软件进行正常读写，也无法通过Python的文件对象直接读取和理解二进制文件的内容。必须正确理解二进制文件结构和序列化规则，才能准确地理解二进制文件内容并且设计正确的反序列化规则。
- 所谓**序列化**，简单地说就是把内存中的数据在不丢失其类型信息的情况下转成对象的二进制形式的过程，对象序列化后的形式经过正确的**反序列化**过程应该能够准确无误地恢复为原来的对象。

✓ Python中常用的**序列化模块有struct、pickle、json、marshal和shelve**。

7.4 二进制文件操作模块

使用pickle模块

使用pickle模块可以实现Python对象的序列化和反序列化。dump()方法会将序列化的对象保存到文件中，而dumps()方法则将序列化的对象保存到内存中。

反序列化对象即将二进制字节流转换为Python对象。可以使用pickle模块的load()和loads()方法实现，两者的区别在于load()方法从文件中加载序列化的对象，而loads()方法从内存中加载序列化的对象。

```
import pickle

a = [1,2,3]
b = pickle.dumps(a)              #dumps()保存到内存
with open(r"d:\temp\t1.txt",'wb') as f:
    pickle.dump(a,f)             #dump()保存到文件

c=pickle.loads(b)                #loads()从内存中读取
mylist=pickle.load(f)            #load()从文件中读取
```

7.4 二进制文件操作模块

- 使用pickle模块

✓ 例： 写入二进制文件。

```
import pickle

i = 13000000
a = 99.056
s = '中国人民123abc'
lst = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
tu = (-5, 10, 8)
coll = {4, 5, 6}
dic = {'a':'apple', 'b':'banana', 'g':'grape', 'o':'orange'}
data = [i, a, s, lst, tu, coll, dic]

with open('sample_pickle.dat', 'wb') as f:
    try:
        pickle.dump(len(data), f) #表示后面将要写入的数据个数
        for item in data:
            pickle.dump(item, f)
    except:
        print('写文件异常!')          #如果写文件异常则跳到此处执行
```

7.4 二进制文件操作模块

- 使用pickle模块

✓ 例：读取二进制文件。

```
import pickle

with open('sample_pickle.dat', 'rb') as f:
    n = pickle.load(f)          #读出文件的数据个数
    for i in range(n):
        x = pickle.load(f)
        print(x)
```

7.4 二进制文件操作模块

- 使用Struct模块

✓ 例：使用struct模块写入二进制文件。

struct按照格式(fmt)，把数据封装成字符串流

```
import struct

n = 1300000000
x = 96.45
b = True
s = 'a1@中国'
sn = struct.pack('if?', n, x, b) #序列化
with open('sample_struct.dat', 'wb') as fp:
    fp.write(sn)           #写入字节串
    fp.write(s.encode()) #字符串直接编码为字节串写入
```

Format	c Type	Python
x	pad byte	no value
c	char	string of length 1
b	signedchar	integer
B	unsignedchar	integer
?	_Bool	bool
h	short	integer
H	unsignedshort	integer
i	int	integer
I	unsignedint	integer or long
l	long	integer
L	unsignedlong	long
q	longlong	long
Q	unsignedlonglong	long
f	float	float
d	double	float
s	char[]	string



7.4 二进制文件操作模块

使用Struct模块

例：使用struct模块读取二进制文件。

```
import struct

with open('sample_struct.dat', 'rb') as fp:
    sn = fp.read(9)
    tu = struct.unpack('if?', sn)
    print(tu)
    n, x, bl = tu
    print('n=', n)
    print('x=', x)
    print('bl=', bl)
    s = fp.read(9).decode()
    print('s=', s)
```

返回一个由解包数据(string)得到的一个元组(tuple)

读取的字节数为啥是9?

Python中encode()函数默认使用UTF-8编码。
encode()`函数用于将字符串编码为指定的编码格式，返回一个字节对象。
decode()`函数用于将字节对象解码为指定的字符串，返回一个字符串。

```
text = "你好"
st = text.encode('utf-8')
print(st)
```

b'\xe4\xbd\xa0\xe5\xa5\xbd’

```
len(st)
```

6

使用Struct模块

- 读取的字节数为啥是9呢，而不是其他数字呢？
  - 原因：

```
>>> import struct
>>> struct.pack('if?', 13000, 56.0, True)
b'\xc82\x00\x00\x00\x00`B\x01'
>>> len(_)
9
#int, float占4字节, bool占1字节
>>> len(struct.pack('if?', 9999, 5336.0, False))
9
>>> x = 'a1@中国'
>>> len(x.encode())
9
```

7.4 二进制文件操作模块

使用marshal序列化

- marshal模块提供了两个主要函数：marshal.dump()和marshal.load()。

```
>>> import marshal                                    #导入模块
>>> x1 = 30                                           #待序列化的对象
>>> x2 = 5.0
>>> x3 = [1, 2, 3]
>>> x4 = (4, 5, 6)
>>> x5 = {'a':1, 'b':2, 'c':3}
>>> x6 = {7, 8, 9}
>>> x = [eval('x'+str(i)) for i in range(1,7)]   #把待序列化对象放到列表中
>>> x
[30, 5.0, [1, 2, 3], (4, 5, 6), {'a': 1, 'b': 2, 'c': 3}, {8, 9, 7}]
>>> with open('test.dat', 'wb') as fp:                #创建二进制文件
    marshal.dump(len(x), fp)                          #先写入对象个数
    for item in x:
        marshal.dump(item,fp)
```



7.4 二进制文件操作模块

DMiC -HFUT-

- 使用marshal序列化

```
>>> with open('test.dat', 'rb') as fp:      #打开二进制文件
    n = marshal.load(fp)                     #获取对象个数
        for i in range(n):
            print(marshal.load(fp))          #反序列化，输出结果


30
5.0
[1, 2, 3]
(4, 5, 6)
{'a': 1, 'b': 2, 'c': 3}
{8, 9, 7}
```

7.5 文件级操作

➢文件级别操作模块

python 提供了2个内置库os模块，shutil模块用于与操作系统交互

os模块不仅提供了新建文件，删除文件，查看文件属性的操作功能，还提供了对文件路径的操作功能。

但是，对于移动，复制、打包、压缩及文件夹功能，os模块没有提供相关的函数，此时用到shutil模块。

7.5 文件级操作

- os模块常用的目录操作函数

方法	功能说明
access(path, mode)	测试是否可以按照mode指定的权限访问文件
chdir(path)	把path设为当前工作目录
chmod(path, mode, *, dir_fd=None, follow_symlinks=True)	改变文件的访问权限
curdir	当前文件夹
environ	包含系统环境变量和值的字典
extsep	当前操作系统所使用的文件扩展名分隔符
get_exec_path()	返回可执行文件的搜索路径
getcwd()	返回当前工作目录
listdir(path)	返回path目录下的文件和目录列表
open(path, flags, mode=0o777, *, dir_fd=None)	按照mode指定的权限打开文件，默认权限为可读、可写、可执行
popen(cmd, mode='r', buffering=-1)	创建进程，启动外部程序



7.5 文件级/目录操作

DMiC
-HFUT-

目录操作示例

```
>>> import os
>>> os.getcwd()                                  #返回当前工作目录
'E:\\tempc'
>>> os.mkdir(os.getcwd()+'\\temp')               #创建目录
>>> os.chdir(os.getcwd()+'\\temp')               #改变当前工作目录
>>> os.getcwd()
'E:\\tempc\\temp'
>>> os.listdir('.')
[]
```

7.5 文件级/目录操作

- os模块常用的文件操作函数

方法	功能说明
remove(path)	删除指定的文件，要求用户拥有删除文件的权限，并且文件没有只读或其他特殊属性
rename(src, dst)	重命名文件或目录，可以实现文件的移动，若目标文件已存在则抛出异常，不能跨越磁盘或分区
replace(old, new)	重命名文件或目录，若目标文件已存在则直接覆盖，不能跨越磁盘或分区
scandir(path='.')	返回包含指定文件夹中所有DirEntry对象的迭代对象，遍历文件夹时比listdir()更加高效
sep	当前操作系统所使用的路径分隔符
startfile(filepath [, operation])	使用关联的应用程序打开指定文件或启动指定应用程序
stat(path)	返回文件的所有属性
system()	启动外部程序
truncate(path, length)	将文件截断，只保留指定长度的内容
write(fd, data)	将bytes对象data写入文件fd

7.5 文件级操作

- os.system(shell)运行shell命令

```
import os
os.system("pause")   #暂停
os.system("ping www.baidu.com")
```

- listdir(path)返回path目录下的所有文件和目录列表

```
import os
lt=os.listdir('C:\Program Files\Tencent')
print(lt)
['QQ', 'QQBrowser']
```

- getcwd()返回当前工作目录

```
print(os.getcwd())
'D:\\temp'
```

7.5 文件级/目录操作

- 示例

```
>>> import os
>>> os.rename('c:\\test1.txt', 'c:\\test2.txt')  # 此时c:\\test1.txt不存在
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    os.rename('c:\\test1.txt', 'c:\\test2.txt')
FileNotFoundError: [WinError 2] 系统找不到指定的文件。: 'c:\\test1.txt' ->
   'c:\\test2.txt'
>>> os.rename('c:\\dfg.txt', 'c:\\test2.txt')    # 可以实现文件的改名和移动
```

7.5 文件级/目录操作

- os.path模块是操作文件和路径最常用的模块

方法	功能说明
abspath(path)	返回给定路径的绝对路径
basename(path)	返回指定路径的最后一个组成部分
commonpath(paths)	返回给定的多个路径的最长公共路径
commonprefix(paths)	返回给定的多个路径的最长公共前缀
dirname(p)	返回给定路径的文件夹部分
exists(path)	判断文件是否存在
getatime(filename)	返回文件的最后访问时间
getctime(filename)	返回文件的创建时间
getmtime(filename)	返回文件的最后修改时间
getsize(filename)	返回文件的大小

7.5 文件级操作

- dirname(path)返回指定路径的目录部分

```
import os
path1 = r'C:\ab\to\file.txt'
print(os.path.dirname(path1))
```

```
C:\ab\to
```

- exists(path)检查指定路径是否存在。

```
import os
path1 = r'C:\ab\to\file.txt'
print(os.path.exists(path1))
```

```
False
```

7.5 文件级/目录操作

➢示例

■ 列出当前目录下所有扩展名为pyc的文件：

```
>>> import os
>>> [fname for fname in os.listdir(os.getcwd()) if
     os.path.isfile(fname) and fname.endswith('.pyc')]
['consts.pyc', 'database_demo.pyc', 'nqueens.pyc']
```

■ 如何列出当前目录下（或指定目录下）所有的图像文件？

```
import os
file_path="./image"  #当前目录中的image子目录
files=os.listdir(file_path)
image_files=[]
for file in files:
    if file.endswith('.jpg') or file.endswith('.jpeg') or
   file.endswith('.bmp'):
        image_files.append(file)
print(image_files)
```

```
['image1.bmp', 'image1.jpg']
```

7.5 文件级/目录操作

- shutil模块，为os模块补充，提供了复制、移动、删除、压缩、解压等操作

方法	功能说明
copy(src, dst)	复制文件，新文件具有同样的文件属性，如果目标文件已存在则抛出异常
copy2(src, dst)	复制文件，新文件具有原文件完全一样的属性，包括创建时间、修改时间和最后访问时间等等，如果目标文件已存在则抛出异常
copyfile(src, dst)	复制文件，不复制文件属性，如果目标文件已存在则直接覆盖
copyfileobj(fsrc, fdst)	在两个文件对象之间复制数据，例如copyfileobj(open('123.txt'), open('456.txt', 'a'))
copymode(src, dst)	把src的模式位（mode bit）复制到dst上，之后二者具有相同的模式
copystat(src, dst)	把src的模式位、访问时间等所有状态都复制到dst上
copytree(src, dst)	递归复制文件夹
disk_usage(path)	查看磁盘使用情况
move(src, dst)	移动文件或递归移动文件夹，也可以给文件和文件夹重命名
rmtree(path)	递归删除文件夹
make_archive(base_name, format, root_dir=None, base_dir=None)	创建tar或zip格式的压缩文件
unpack_archive(filename, extract_dir=None, format=None)	解压缩压缩文件

7.5 文件级/目录操作

- shutil模块
  - 使用该模块的copyfile()方法复制文件

```
>>> import shutil
>>> shutil.copyfile('C:\\dir.txt', 'C:\\dir1.txt')
```

  - 将C:\Python\Dlls文件夹以及该文件夹中所有文件压缩至D:\a.zip文件

```
>>> shutil.make_archive('D:\\a', 'zip', 'C:\\Python', 'Dlls')
```

参数'D:\\a'：表示压缩后的文件名
参数'zip'：为压缩的格式，有tar、zip 等
参数'C:\\Python'：打包时切换到的根路径。默认值为当前路径
参数'Dlls'：要打包的路径，对某目录进行打包



7.6 应用案例

DMiC -HFUT-

✓ 例： 计算文本文件中最长行的长度。

- 方法一：

```
with open('d:\\test.txt','r') as f:
    allLineLens = [len(line.strip()) for line in f]
print(max(allLineLens))
```

- 方法二：

```
with open('d:\\test.txt','r') as f:
    print(max(len(line.strip()) for line in f))
```

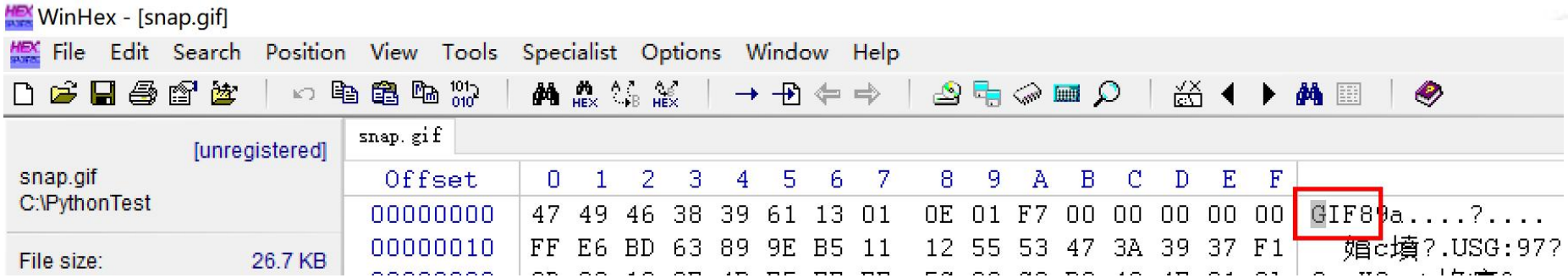
7.6 应用案例

✓ 例：判断一个文件是否为GIF图像文件。

```
>>> def is_gif(fname):
    with open(fname, 'rb') as fp:
        first4 = fp.read(4)
    return first4 == b'GIF8'

>>> is_gif('a.gif')
True
>>> is_gif('a.png')
False
```







7.6 应用案例

DMiC
-HFUT-

使用Python中的difflib库进行自动查重

difflib库中最常用的两个类是SequenceMatcher和Differ，它们都可以用来比较两个序列（如字符串、列表、元组等）之间的差异，并生成相应的结果。

SequenceMatcher类可以用来计算两个序列之间的相似度，以及找出它们最长的匹配子序列。

Differ类可以用来生成两个序列之间的差异报告，以便于人类阅读和理解。

```
import difflib
with open('texta.txt','r')as f:
    a=f.read()
with open('textb.txt','r')as f:
    b=f.read()
sm=difflib.SequenceMatcher(None,a,b)
similarity=sm.ratio()  #打印两篇文章的相似度
print(f'相似度:{similarity:.2f}')
```

7.6 应用案例

✓ 例： 比较两个文本文件内容是否相同。

```
import difflib
A = open('c:\\dir.txt','r')        #python的标准库模块
B = open('c:\\dir1.txt','r')
contentA = A.read()
contentB = B.read()
s = difflib.SequenceMatcher(None,contentA,contentB)
result = s.get_opcodes()
#get_opcodes()返回一个列表，包含了将第一个序列转换为第二个序列所需的操作
for tag,i1,i2,j1,j2 in result:
    print("%s contentA[%d:%d]=%s contentB[%d:%d]=%s"%\
          (tag,i1,i2, contentA [i1:i2],j1,j2, contentB[j1:j2]))
```

```
equal contentA[0:8]=Python是一 contentB[0:8]=Python是一
replace contentA[8:9]=种 contentB[8:12]=门通用的
equal contentA[9:15]=高级编程语言 contentB[12:18]=高级编程语言
replace contentA[15:16]=， contentB[18:19]=。
equal contentA[16:17]=它 contentB[19:20]=它
```

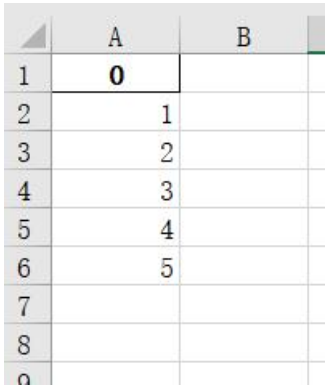
7.6 应用案例

Excel文件处理，常用的第三方库：

- pandas：用于数据分析和处理，支持读写Excel文件。
- openpyxl：用于读写Excel文件。
- xlrd：用于读取Excel文件，xlwt：用于写入Excel文件。

```
import pandas as pd  #pandas模块要提前pip install pandas
   #使用pandas库可以将Excel文件读取到DataFrame对象中
data = pd.read_excel("data.xlsx")
  #如Excel文件有多个Sheet，通过指定Sheet名称或索引来读取指定的Sheet。
data = pd.read_excel("data.xlsx", sheet_name="Sheet1")
print(data.values[2])  #输出第3行数据

data=[1,2,3,4,5]
   # 创建DataFrame
df = pd.DataFrame(data)
   # 写入Excel文件
df.to_excel("output.xlsx", index=False)
#index是否写入行索引，默认为 True
```



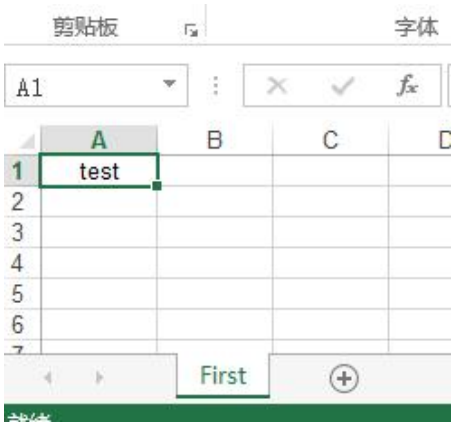
	A	B
1	0	
2	1	
3	2	
4	3	
5	4	
6	5	
7		
8		

7.6 应用案例

✓ 例：使用xlwt模块写入Excel文件。

```
from xlwt import * # xlwt模块要提前pip install xlwt

book = Workbook()                          #新建workbook
sheet1 = book.add_sheet("First")           #新建worksheet
al=Alignment()
al.horz=Alignment.HORZ_CENTER
al.vert=Alignment.VERT_CENTER              #对齐方式
borders=Borders()
borders.bottom=Borders.THICK               #边框样式
style=XFStyle()
style.alignment=al
style.borders=borders
row0=sheet1.row(0)
row0.write(0,'test',style=style)           #写入单元格
book.save(r'd:\test.xls')                  #保存文件
```





7.6 应用案例

DMIC
-HFUT-

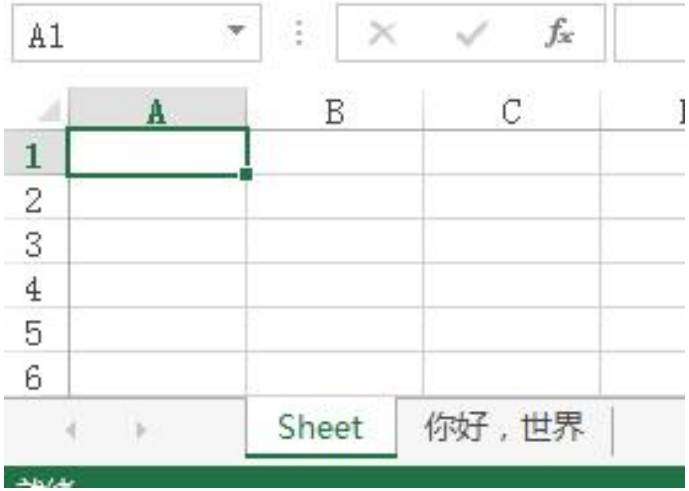
✓ 例：使用xlrd读取Excel文件。

```
>>> import xlrd
>>> book = xlrd.open_workbook(r'd:\test.xls')
>>> sheet1 = book.sheet_by_name('First')
>>> row0 = sheet1.row(0)
>>> print(row0[0])
text:'test'
>>> print(row0[0].value)
test
```

7.6 应用案例

✓ 例：openpyxl模块读写Excel文件。

```
import openpyxl
from openpyxl import Workbook
fn = r'f:\test.xlsx'                                  #文件名
wb = Workbook()                                       #创建工作簿
ws = wb.create_sheet(title='你好，世界')               #创建工作表
ws['A1'] = '这是第一个单元格'                          #单元格赋值
ws['B1'] = 3.1415926
wb.save(fn)                                           #保存Excel文件
```



7.6 应用案例

```
wb = openpyxl.load_workbook(fn)          #打开已有的Excel文件
ws = wb.worksheets[1]                    #打开指定索引的工作表
print(ws['A1'].value)                    #读取并输出指定单元格的值
ws.append([1,2,3,4,5])                   #添加一行数据
ws.merge_cells('F2:F3')                  #合并单元格
ws['F2'] = "=sum(A2:E2)"                 #写入公式
for r in range(10,15):
    for c in range(3,8):
        ws.cell(row=r, column=c, value=r*c) #写入单元格数据
wb.save(fn)
```

	A	B	C	D	E	F	G
1	这是第一个单元格	3.141593					
2	1	2	3	4	5		
3						15	
4							
5							
6							
7							
8							
9							
10			30	40	50	60	70
11			33	44	55	66	77
12			36	48	60	72	84
13			39	52	65	78	91
14			42	56	70	84	98
15							



7.6 应用案例

- 例：把记事本文件test.txt转换成Excel 2007+文件。test.txt文件中第一行为表头，从第二行开始是实际数据，并且表头和数据行中的不同字段信息都是用逗号分隔。

```
from openpyxl import Workbook

def main(txtFileName):
    new_XlsxFileName = txtFileName[:-3] + 'xlsx'
    wb = Workbook()
    ws = wb.worksheets[0]
    with open(txtFileName) as fp:
        for line in fp:
            line = line.strip().split(',')
            ws.append(line)
    wb.save(new_XlsxFileName)

main('test.txt')
```

7.7 本章小结

内容回顾

- 文件操作的相关概念
- 两种文件形式及其操作模块
- 数据组织形式
- 实际应用